

Risposte parte A

Domanda 1

Architettura dei sistemi operativi per sistemi di elaborazione monoprocesso: si presentino le principali architetture dei sistemi operativi, descrivendone caratteristiche, vantaggi e limiti.

1. Sistema monolitico - struttura semplice

È la struttura più comune, adottata dai sistemi UNIX originali e dalle prime versioni di Linux

- **caratteristiche:** non c'è una vera e propria struttura divisa; l'intero SO - kernel è composto da un unico file binario statico che gira in un singolo spazio di indirizzamento. Il kernel offre (file system, scheduling CPU, gestione della memoria) direttamente tramite chiamate di sistema.
- **vantaggi:** massima efficienza prestazionale (minimo overhead¹ nelle chiamate di sistema)
- **svantaggi:** codice difficile da comprendere e mantenere, difficile da estendere (*tightly coupled*), una modifica in una parte del sistema può avere effetti imprevisti su altre parti, rendendo il debugging complesso.

2. Sistema con struttura gerarchica

Rappresenta un primo passo verso l'organizzazione logica rispetto al monolitico

- **caratteristiche** si basa sull'identificazione delle dipendenze gerarchiche tra le varie funzioni del SO. Le funzioni sono raggruppate in base a chi chiama chi.
- **vantaggi:** migliore organizzazione logica rispetto al monolitico
- **svantaggi** la manutenzione rimane complessa se non vi è un rigido incapsulamento.

3. sistema stratificato - *layered*

È la formalizzazione del sistema gerarchico, il sistema viene suddiviso in livelli (strati) sovrapposti, dallo stato 0 (hardware) allo strato N (interfaccia utente).

- **caratteristiche:** ogni strato è un oggetto che incapsula dati e operazioni. La regola fondamentale è che uno strato i può usare solo le funzioni e i servizi del livello immediatamente inferiore ($i - 1$) e fornire servizi a quello superiore ($i + 1$).
- **vantaggi:** modularità, debugging, il sistema è facile da verificare, si testa il livello 0, dopodiché il livello 1 e così via, se si verifica un errore, la causa sta nel livello in analisi (perché il livello precedente è già stato verificato).
- **svantaggi:**
 - efficienza, ogni chiamata dell'utente deve attraversare molteplici strati, aggiungendo overhead e riducendo le prestazioni rispetto al monolitico
 - progettazione, definire esattamente in quale strato posizionare una funzione è difficile (es. il driver del disco dipende dalla memoria, o la gestione della memoria dipende dal driver del disco?).

4. sistema a microkernel

Nasce per risolvere la crescita incontrollata delle dimensioni del kernel nei sistemi stratificati e monolitici.

- **caratteristiche:** si muovono tutte le componenti non essenziali (come File System o driver complessi), spostandole nello "spazio utente" sotto forma di servizi/processi di sistema. Il kernel (ridotto al minimo - *micro*), gestisce solo le funzioni vitali: comunicazioni tra processi (IPC), gestione base della memoria e scheduling della CPU.
- **vantaggi:**
 - affidabilità, se un servizio (es. driver video) fallisce, va in crash solo quel processo, non l'intero sistema operativo
 - estendibilità, aggiungere nuovi servizi non richiede la ricompilazione del kernel,
- **svantaggi:**
 - prestazioni, l'interazione tra servizi utente richiede un continuo scambio di messaggi attraverso il microkernel (context-switch²), rallentando l'esecuzione rispetto alle chiamate di funzione dirette.

5. sistema a moduli funzionali - caricabili

È l'approccio moderno che combina vantaggi del monolitico e del microkernel

- **caratteristiche:** il kernel possiede un nucleo di base fisso, ma può collegare dinamicamente servizi aggiuntivi (moduli) sia all'avvio che durante l'esecuzione
- **vantaggi:**
 - flessibilità, simile al microkernel (si caricano solo i driver/servizi necessari)
 - prestazioni del monolitico, i moduli, una volta caricati, diventano parte del kernel e comunicano direttamente senza l'overhead dello scambio di messaggi tipico del microkernel

6. sistema a macchine virtuali

Approccio in cui il sistema operativo astrae l'HW per creare diverse istanze di esecuzione, oggi alla base del cloud.

- **caratteristiche:** il kernel (spesso chiamato Hypervisor) crea un'illusione: presenta a più processi o sistemi operativi "ospiti" (Guest) un'interfaccia che sembra dedicato. L'HW fisico è condiviso, ma ogni VM crede di avere la sua CPU, il suo disco, ecc.
- **vantaggi**
 - isolamento totale: se una VM crasha o viene infettata da un virus le altre e l'host non ne risentono
 - compatibilità: permette di fare girare più OS diversi sulla stessa macchina fisica contemporaneamente.
- **limiti** overhead dovuto alla simulazione e gestione delle risorse condivise da parte dell'Hypervisor.

¹overhead - spreco di risorse (tempo della CPU o memoria) che il sistema impiega per gestire la propria struttura interna (burocrazia), invece che per eseguire i programmi dell'utente. Più strati o passaggi ci sono, maggiore è l'overhead.

²context-switch - operazione costosa di salvataggio dello stato del processo corrente e caricamento del successivo necessaria per passare l'uso della CPU da un processo all'altro. Durante questo scambio, la CPU non esegue calcoli utili.

Domanda 2

La schedulazione dei processi in un sistema di elaborazione monoprocesso: si descrivano le principali tecniche per realizzare la schedulazione dei processi in un sistema monoprocesso, evidenziandone le caratteristiche.

La schedulazione è il meccanismo fondamentale per gestire l'assegnazione della CPU ai processi, con l'obiettivo di massimizzare l'utilizzo del processore e garantire un'esecuzione equa ed efficiente.

1. Contesto e modalità di attivazione

- Livelli: la schedulazione avviene a tre livelli
 - a lungo termine - componente che decide quali *job* ammettere dal disco alla memoria centrale per essere eseguiti; il suo compito principale è controllare il grado di **multiprogrammazione** (numero di processi attivi) per bilanciare il carico del sistema.
 - a medio termine - gestisce lo **swapping** rimuovendo temporaneamente dalla memoria i processi inattivi o bloccati per liberare le risorse, reintroducendoli quando possibile.
 - a breve termine - selezione del prossimo processo da eseguire dalla memoria centrale

le tecniche di seguito descritte riguardano principalmente lo scheduler a breve termine.

- modalità
 - non pre-emptive (senza rilascio anticipato): il processo mantiene la CPU finché non termina o richiede I/O
 - pre-emptive: il sistema può interrompere forzatamente un processo in esecuzione (es. per scadenza di tempo o priorità)

Lo scheduler, in generale, può ordinare i processi in base ad alcuni criteri, quali l'utilizzo del processore, la frequenza di completamento, il tempo di completamento, il tempo d'attesa o il tempo di risposta. Nello specifico, però, esistono diverse politiche di schedulazione:

2. Principali politiche (algoritmi)

a. first come first served

l'algoritmo più semplice: la CPU viene assegnata al processo che per primo ne fa richiesta

- **caratteristiche** rigorosamente non pre-emptive, salvo terminazione o blocco volontario
- **svantaggio** l'effetto convoglio¹ causato se un processo lungo (CPU-Bound) occupa il processore, tutti gli altri (anche quelli brevi o I/O Bound) devono attendere in coda inutilmente, aumentando drasticamente il *tempo di attesa medio*.

b. shortest job first

l'algoritmo seleziona, tra i processi in attesa, quello che richiede il minor tempo di utilizzo della CPU (*CPU burst² più breve*) per la sua prossima esecuzione

- **caratteristiche** può essere non pre-emptive o pre-emptive (Shortest Remaining Time First), dove se arriva un processo con burst inferiore al residuo di quello corrente, avviene la sostituzione,
- **vantaggi** è dimostrato essere ottimale per minimizzare il tempo di attesa medio.
- **svantaggi** è difficile da implementare perché non si conosce a priori la durata del prossimo CPU burst, è necessario stimarla usando una media esponenziale delle esecuzioni passate.

c. **schedulazione con priorità**

a ogni processo è associato un numero (priorità) e la CPU è assegnata a quello con priorità maggiore (solitamente definita da fattori interni o esterni, ad esempio *processi di sistema > processi utente*)

- **svantaggi** il problema principale è la starvation (attesa indefinita), processi a bassa priorità potrebbero non essere mai eseguiti se c'è un flusso continuo di processi ad alta priorità
- **soluzione** si applica l'aging, ovvero si aumenta gradualmente la priorità di un processo che attende da molto tempo

d. **round robin**

per sistemi time-sharing, è simile al FCFS ma con prelazione basata su quanto di tempo assegnato ad ogni processo.

- **caratteristiche** la coda è circolare, se il processo esaurisce il suo quanto, viene interrotto e messo in fondo alla coda.
- **svantaggi** se il quanto di tempo è troppo grande l'algoritmo degenera in un FCFS, se il quanto è troppo piccolo c'è un eccessivo overhead dovuto ai frequenti context-switch

e. **coda a più livelli**

i processi vengono classificati in gruppi (es. foreground/interattivi, background/batch) e assegnati in modo permanente a code distinte

- **caratteristiche** ogni coda ha il proprio algoritmo ed esiste anche una priorità tra code stesse (una coda batch viene servita solo se quella interattiva è vuota) oppure una suddivisione del tempo CPU garantita

f. **coda a più livelli con retroazione**

tecnica più generale e complessa, a differenza della precedente permette ai processi di **spostarsi tra le code** in base al loro comportamento dinamico (CPU burst)

- **caratteristiche** un processo entra nella coda ad alta priorità (quanto di tempo breve, es. 8 ms. Se non finisce in tempo viene "degradato" a una coda a priorità inferiore con quanto più lungo (es. 16 ms), fino ad arrivare alla coda FCFS
- **vantaggi** favorisce i processi interattivi e brevi, regolando i processi CPU-bound nelle code a bassa priorità, prevenendo le starvation tramite meccanismi di aging

¹effetto convoglio: fenomeno di inefficienza tipico dell'algoritmo FCFS dove un unico processo "lento" blocca tutti gli altri "veloci".

²burst: intervallo di tempo continuo in cui un processo utilizza attivamente la CPU per eseguire istruzioni, senza interruzioni.

Domanda 3

La sincronizzazione tra processi in un sistema di elaborazione monoprocessore: si descrivano le principali tecniche per realizzare la sincronizzazione tra processi in un sistema monoprocessore, evidenziandone le caratteristiche.

In un sistema con processi cooperanti, l'accesso concorrente ai dati condivisi può causare incoerenze note come **Race Condition**. Per garantire l'integrità dei dati, il sistema operativo deve sincronizzare i processi proteggendo la **Sezione Critica** (la parte di codice che accede alla risorsa condivisa).

Le soluzioni si basano su due concetti logici fondamentali, che vengono poi implementati tramite specifiche tecniche hardware o software:

1. **Variabile di Turno:** È una variabile che definisce esplicitamente a chi tocca l'uso della risorsa (sincronizzazione stretta). È la base logica per algoritmi di alternanza, ma è poco flessibile.
2. **Variabile di Lock:** È una variabile di stato (es. `lock = 0` libera, `lock = 1` occupata). Quando un processo vuole entrare, controlla questa variabile: se libera, la setta a "occupata" ed entra; altrimenti attende.
 - *Il problema fondamentale:* La lettura e modifica della variabile di lock devono essere atomiche. Per farlo, si usano le tecniche descritte di seguito.

Le principali tecniche implementative si suddividono in **Approcci Hardware** (basso livello) e **Strumenti Software** (alto livello).

A. Approcci Hardware (Basso Livello)

Queste tecniche servono a garantire l'atomicità necessaria per manipolare le variabili di lock o proteggere la sezione critica direttamente.

1. Disabilitazione degli Interrupt

È la tecnica specifica per eccellenza dei **sistemi monoprocessore**.

- **Funzionamento:** Poiché in un monoprocessore l'unico modo per interrompere un processo è un interrupt (o la fine del quanto di tempo), se si disabilitano le interruzioni prima di entrare nella sezione critica e le si riabilitano all'uscita, si garantisce che l'istruzione non venga fermata a metà.
- **Vantaggio:** Atomicità garantita in modo semplice senza strumenti complessi.
- **Limiti:**
 - **Pericoloso:** Concesso solo al Kernel (se un processo utente lo facesse e andasse in loop, il sistema morirebbe).
 - **Inutilizzabile su Multiprocessore:** Disabilitare l'interrupt su una CPU non ferma le altre.

2. Istruzioni HW Speciali (Atomic Instructions)

I processori moderni offrono istruzioni per manipolare le **variabili di lock** in modo sicuro (atomico) senza dover disabilitare gli interrupt.

- **Istruzioni principali:**
 - `test_and_set()` : Legge, restituisce il valore attuale e imposta a *true* in un passo indivisibile.
 - `compare_and_swap()` (CAS): Scambia il valore solo se corrisponde a un'attesa.
- **Ruolo:** Sono i "mattoni" fondamentali usati per costruire gli strumenti software come i Mutex.

B. Strumenti Software (Alto Livello)

Questi strumenti offrono astrazioni più gestibili per i programmatori.

3. Lock Mutex (Implementazione della Variabile di Lock)

È l'implementazione software diretta del concetto di **Variabile di Lock** descritto all'inizio.

- **Funzionamento:** Usa una variabile booleana `available`.
 - `acquire()` : Controlla se il lock è libero. Se non lo è (`!available`), attende. Se è libero, lo prende (setta `available = false`).
Nota: Questa operazione usa internamente istruzioni atomiche o disabilitazione interrupt per essere sicura.
 - `release()` : Rilascia il lock (`available = true`).
- **Caratteristiche:**
 - **Spinlock (Busy Waiting):** Nell'implementazione base, se il lock è occupato, il processo gira in un ciclo `while` .
 - **Svantaggio su Monoprocessore:** Gli spinlock sono **inefficienti** su singola CPU. Se il processo A ha il lock ed è sospeso, e il processo B gira a vuoto controllando il lock, B spreca solo tempo CPU che servirebbe ad A per finire e liberare la risorsa.

4. Semafori

Strumenti più evoluti che superano il concetto semplice di lock "aperto/chiuso" e il problema del busy waiting.

- **Differenza dal Lock:** Non hanno un "proprietario". Un processo può fare l'attesa (`wait`) e un altro diverso può inviare il segnale (`signal`).
- **Gestione Attesa:** Invece di girare a vuoto (busy waiting), usano `block()` (il processo va in coda di attesa e rilascia la CPU) e `wakeup()` (viene risvegliato quando la risorsa si libera). Questo è molto più efficiente su monoprocessore.
- **Operazioni:**
 - `wait(S)` : Decrementa S. Se $S < 0$, blocca il processo (coda).
 - `signal(S)` : Incrementa S. Se $S \leq 0$, sveglia un processo dalla coda.

5. Monitor

Costrutti di alto livello (ADT) che incapsulano dati e procedure, gestendo la mutua esclusione automaticamente (senza gestione esplicita di lock da parte del programmatore).

- **Gestione Sincronizzazione:** Poiché il lock automatico non basta per la cooperazione (es. "aspetto il mio turno"), si usano **Variabili Condition** (x, y):
 - `x.wait()` : Il processo si sospende e **rilascia il Monitor** (rilascia il lock globale del monitor).
 - `x.signal()` : Risveglia un processo sospeso su x .
- **Semantica del Segnale:** Risolve il conflitto di avere due processi attivi nel monitor dopo una signal:
 - *Signal and Wait:* Chi segnala si ferma e lascia passare chi è stato svegliato.
 - *Signal and Continue:* Chi segnala continua fino alla fine, chi è svegliato aspetta che il monitor si liberi.

Domanda 4

La comunicazione tra processi in un sistema di elaborazione monoprocesso: si descrivano le principali tecniche per realizzare la comunicazione tra processi in un sistema monoprocesso, evidenziandone le caratteristiche.

La comunicazione tra processi (IPC - *Interprocess Communication*) è essenziale per consentire ai processi cooperanti di scambiarsi dati e sincronizzarsi. In un sistema **monoprocesso**, la comunicazione avviene alternando l'esecuzione dei processi tramite *Context Switch*.

Le tecniche si possono classificare in base alla modalità di connessione: **Diretta** (comunicazione esplicita tra processi) e **Indiretta** (comunicazione mediata da un'entità terza).

1. IPC Diretti

In questo modello, i processi che vogliono comunicare devono riferirsi esplicitamente l'uno all'altro o accedere direttamente a risorse comuni.

- **Memoria Condivisa (Shared Memory)**

mittente e ricevente condividono un'area nel loro spazio dei dati globali. Ciò significa che, se quello spazio viene modificato in uno dei due processi, deve esserlo anche nell'altro. Per rendere ciò possibile bisogna innanzitutto rendere quell'area di memoria a mutua esclusione, in modo da non leggere dati inconsistenti o interrompere una lettura. Inoltre, bisogna realizzare la condivisione, che può essere attuata in due modi. Il primo consiste nell'usare un'area nello spazio riservato al sistema operativo in cui lui stesso si incarica di mantenere copiate le due zone dei processi, andando ad aggiornarle in caso di cambiamenti da parte di una delle due. Il secondo metodo consiste nell'utilizzare un'area fisicamente condivisa tra i due processi.

- **Condivisione di buffer**

mittente e ricevente utilizzano uno spazio di indirizzamento come buffer per inviare e prelevare le informazioni che si devono scambiare. Si crea quindi un rapporto produttore-consumatore tra i due processi

- **Scambio di Messaggi (Diretto)**

- **Caratteristiche:** I processi comunicano scambiandosi pacchetti di dati gestiti dal Kernel, senza condividere memoria.
- **Funzionamento:** Richiede che mittente e destinatario si nominino esplicitamente.
 - `send(P, message)` : Invia un messaggio al processo P.
 - `receive(Q, message)` : Riceve un messaggio dal processo Q.
- **Vantaggi:** Facile da implementare per piccole moli di dati; evita i conflitti di memoria.
- **Svantaggi:** Meno efficiente della memoria condivisa, poiché ogni operazione richiede l'intervento del Kernel (System Call) e copie dei dati.

2. IPC Indiretti

In questo modello, i messaggi non sono inviati direttamente da un processo all'altro, ma passano attraverso un contenitore o un canale intermedio. Questo disaccoppia il mittente dal destinatario.

Mailbox (Caselle di Posta)

- **Caratteristiche:** I messaggi sono inviati e ricevuti da una "casella" (o porta) identificata da un ID univoco, non dal nome del processo.
- **Funzionamento:**
 - `create(M); Delete(M)` creazione ed eliminazione della mailbox.
 - `send(A, message)` : Invia alla mailbox A.
 - `receive(A, message)` : Preleva dalla mailbox A.
- **Vantaggi:** Flessibilità (**Molti-a-Molti**). Più processi possono scrivere nella stessa mailbox e più processi possono leggere, permettendo schemi complessi (es. un server che riceve richieste da vari client su un'unica porta).

File e Pipe

rappresentano metodi di comunicazione orientati al flusso di dati (stream)

- File Condivisi: i processi comunicano leggendo e scrivendo su regolari file di disco. La sincronizzazione è gestita tramite meccanismi di locking del file system
- Pipe: sono "file virtuali" in memoria centrale, gestiti come code FIFO. Un processo scrive ad un'estremità e l'altro legge dall'altra in modo sequenziale. Sono ideali per collegare l'output di un comando all'input di un altro, ma impongono un flusso unidirezionale e spesso richiedono una parentela tra processi, oltre ad essere volatili

- **Named Pipe:** esistono permanentemente nel filesystem come file speciali e permettono la comunicazione tra processi non correlati

Socket

- **Caratteristiche:** Definiti come "punti terminali per la comunicazione". I processi mittente e ricevente si trovano su due macchine diverse, la comunicazione avviene attraverso la rete.
- **Struttura:** Un socket è identificato da **Indirizzo IP : Numero di Porta** (es. **127.0.0.1:8080**).
- **Modello:** Client-Server. Utilizzano flussi di byte non strutturati. Il SO gestisce interamente il buffering e la trasmissione dei pacchetti tra i due endpoint.

Domanda 5

Il deadlock in un sistema di elaborazione monoprocesso: se ne dia la definizione, si descrivano le condizioni sotto le quali si verifica, e si indichino le principali tecniche per il suo trattamento, evidenziandone le caratteristiche, vantaggi e limiti.

In un sistema di elaborazione con risorse finite, il Deadlock (o stallo) è la situazione in cui un insieme di processi è bloccato perché ogni processo attende un evento che solo un altro processo dello stesso insieme può generare. Si verifica **sse** sussistono simultaneamente quattro condizioni:

1. mutua esclusione esiste almeno una risorsa che non è condivisibile (es. una zona di memoria o una stampante), se un altro processo la richiede, deve attendere che venga rilasciata.
2. possesso e attesa un processo possiede già almeno una risorsa e, contemporaneamente, è in attesa di acquisirne altre che sono attualmente possedute da altri processi.
3. assenza di pre-emption le risorse non possono essere sottratte forzatamente al processo che le detiene; possono essere rilasciate solo volontariamente dal processo stesso al termine del suo compito.
4. attesa circolare Esiste un ciclo nel grafo di assegnazione delle risorse ($P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_0$).

Per gestire il problema esistono tre approcci principali (oltre a quello di ignorarlo, tipico di Unix/Windows):

1. Prevenzione del deadlock

questa tecnica consiste nel progettare il sistema in modo da invalidare a priori almeno una delle quattro condizioni definitorie, rendendo lo stallo strutturalmente impossibile.

- **negare “possesso e attesa”** si impone al processo di richiede tutte le risorse necessarie all’inizio dell’esecuzione, svantaggio: bassa inefficienza e spreco di risorse allocate ma non usate subito
- **negare “mutua esclusione”** impossibile per risorse intrinsecamente non condivisibili (es. stampante).
- **negare “attesa circolare”** Si impone un **ordinamento numerico crescente** delle risorse. Un processo può richiedere risorse solo in ordine crescente.
- **negare “assenza di pre-emption”** se un processo che detiene risorse ne richiede un’altra che non è disponibile, tutte le risorse che possedeva vengono sottratte dal sistema operativo, oppure il processo viene sospeso e potrà ripartire solo quando riotterrà sia le vecchie risorse che quella nuova.
- **vantaggi**
nessun overhead di calcolo a runtime.

2. Evitare il deadlock

Il sistema decide dinamicamente ad ogni richiesta se concedere la risorsa o far attendere il processo, per evitare di entrare in una situazione pericolosa.

- **requisito fondamentale** ogni processo deve dichiarare anticipatamente il numero massimo di risorse di cui avrà bisogno durante la sua esecuzione.
- questa tecnica mira a mantenere il sistema in uno stato sicuro. Uno stato è sicuro se esiste almeno una sequenza sicura $\langle P_1, P_2, \dots P_n \rangle$ tale che ogni processo P_i possa terminare utilizzando le risorse libere attuali più quelle che verranno rilasciate dai processi precedenti (P_j , con $j < i$) una volta finiti. Se tale sequenza non esiste, lo stato è insicuro e il deadlock possibile.

Il sistema sceglie come implementare la rilevazione a seconda del tipo della tipologia di risorsa:

- per risorse a istanza singola (grafo di allocazione) si estende il grafo delle risorse introducendo gli *archi di prenotazione*, che indicano richieste future. Quando un processo richiede una risorsa, l'arco diventa di *richiesta*. Il sistema concede la risorsa solo se la trasformazione dell’arco non crea cicli nel grafo.

- per risorse a istanza multiple (algoritmo del banchiere), i grafi non bastano. Si usano strutture dati matriciali (*Available, Need, Allocation, Max*). Quando arriva una richiesta, il sistema esegue una simulazione:
 1. verifica che la richiesta non superi il *Need* e l'*Available*.
 2. simula l'assegnazione delle risorse
 3. esegue l'algoritmo di sicurezza: se nello stato simulato esiste ancora una sequenza sicura che porta alla terminazione dei processi, la richiesta viene approvata effettivamente; altrimenti il processo deve attendere per non rendere il sistema insicuro.

Questo approccio garantisce una maggiore efficienza nell'uso delle risorse rispetto alla prevenzione, ma introduce un notevole overhead computazionale, dovendo eseguire un algoritmo complesso ad ogni singola richiesta di risorsa.

3. Rilevazione e ripristino

Il sistema non previene né evita lo stallo, ma ne monitora lo stato in modo da rilevarlo e correggerlo.

- **rilevazione** periodicamente, il SO esamina il grafo di assegnazione alla ricerca di cicli o deadlock.
- **ripristino** una volta rilevato, il deadlock va rotto: tramite la terminazione dei processi dove si abortiscono tutti i processi coinvolti o si seleziona una "vittima" alla volta finché il ciclo non si rompe oppure la prelazione dove si sottraggono forzatamente risorse ad un processo (facendo rollback del suo stato) per darle ad altri.

Anche in questo caso, si ha una maggiore efficienza nell'uso delle risorse rispetto alla prevenzione ma, un alto costo computazionale (overhead) per eseguire l'algoritmo ad ogni richiesta.

Approccio dello struzzo

Esiste un quarto metodo, adottato da Windows e Linux: ignorare il problema lasciando all'utente il compito di terminare i processi bloccati, poiché i deadlock sono rari e le soluzioni software troppo costose.